

---

Machine Learning Based Analog Circuit Fault Diagnosis  
PDF 19 — Chapter 26: day14\_train\_svm.py Explained Line-by-Line  
Baseline SVM Training | Prediction & Evaluation Code

---

# PART 1 — Baseline SVM Training

Chapter 26 — Complete Python Source Code Explained Part 1 — day14\_train\_svm.py (Baseline SVM Training) (Based on your actual source code)

## Before Starting

Congratulations. Until now, our project has been preparing the data. We have Generated the dataset. Added realistic noise. Extracted 17 features. Now, for the first time,

## Machine Learning

enters the project. This chapter explains how your first SVM model is trained. Goal of This Script Professor asks What is the purpose of day14\_train\_svm.py?

## Perfect Answer

This script loads the extracted feature dataset, preprocesses it, trains a baseline Support Vector Machine classifier, evaluates its performance on unseen test data, and provides the initial benchmark before hyperparameter optimization.

## Complete Pipeline

Read CSV

## Separate Features & Labels

## Encode Labels

## Train-Test Split

**StandardScaler** — Train SVM. **Predict** — Evaluate. Everything happens inside this script.

## Import Section

Your code imports libraries from NumPy, Pandas, and Scikit-Learn. Question. Why Scikit-Learn? Because it provides ready-made implementations for SVM StandardScaler Train-Test Split Accuracy Confusion Matrix Classification Report Instead of implementing these algorithms from scratch, we use a well-tested library. Reading the Dataset Your code loads the feature dataset using `pd.read_csv(.)` Question. What does this file contain? Not waveforms. It contains

## 17 Features

+

## Fault Label

Exactly the output of Day 13.

### PROFESSOR ASKS

**Why doesn't the SVM use the original waveform dataset?**

### EXCELLENT ANSWER

The SVM is trained on the extracted feature dataset because the raw waveforms have already been transformed into compact numerical feature vectors that better represent the circuit characteristics. Separating Features and Labels Your code creates two variables. Conceptually **X** — Features. Y

## Fault Labels

Question. Why separate them? Because

## Machine Learning

needs **Inputs** — Features. **Outputs** — Labels. Example Peak RMS Mean. **DC Level** — RC\_Open. The SVM learns this mapping.

## Label Encoding

Your labels look like Normal Open Short Noise Drift Question. Can SVM understand text? No. The classifier requires numbers. Therefore, your code uses a Label Encoder. Conceptually, **Normal** — 0. **Open** — 1. **Short** — 2. **Noise** — 3. **Drift** — 4.

### PROFESSOR ASKS

Why is Label Encoding required?

### EXCELLENT ANSWER

Machine learning algorithms operate on numerical values. Label Encoding converts textual fault names into numerical class identifiers while preserving the class mapping.

## Train-Test Split

Now your code uses `train_test_split(.)` Question. Why? Because we must evaluate the model on data it has never seen. Suppose the dataset contains 1000 samples. Typical split **Training** — 800. **Testing** — 200. Training teaches the model. Testing evaluates the model.

### PROFESSOR ASKS

Why can't we train and test on the same data?

### EXCELLENT ANSWER

Testing on the same data used for training would only measure memorization rather than generalization. A separate test set provides a realistic estimate of model performance on unseen samples.

## Random State

Your code specifies a `random_state` Question. Why? Because without it, every execution creates a different train-test split. Using a fixed random state makes experiments reproducible. Exactly like the random seed used during noise generation. **StandardScaler** One of the most important steps. Your code creates `StandardScaler()` Question. Why? Look at two features.

## Rise Time

0.0008

## Signal Energy

2500 Question. Which feature would dominate distance calculations? Signal Energy. Not because it's more important, but because its numerical values are much larger. Scaling solves this problem. How **StandardScaler** Works For every feature, it computes

## New Value

=

## (Value – Mean)

## Standard Deviation

After scaling, every feature has approximately **Mean** — 0.

## Standard Deviation

1 Now all features are on comparable scales.

### PROFESSOR ASKS

## Why is StandardScaler necessary for SVM?

### EXCELLENT ANSWER

SVM relies on distance calculations to construct the decision boundary. Without feature scaling, variables with larger numerical ranges would dominate the optimization process, reducing classification performance. Fit vs Transform Your code uses `fit_transform()` on training data and `transform()` on testing data. Question. Why not fit both datasets? Because that would leak test information into training. We learn the scaling parameters only from the training set. Then apply the same transformation to the test set.

### PROFESSOR ASKS

## Why not fit the scaler on the test set?

### EXCELLENT ANSWER

The test set must remain completely unseen during training. Fitting the scaler on test data would introduce data leakage and produce overly optimistic evaluation results. Creating the SVM Now your code creates the classifier. Conceptually, `SVC(.)` At this stage, this is the baseline model. No `GridSearchCV`. No hyperparameter tuning. Just the initial model. Question. Why build a baseline first? Because we need something to compare future improvements against. Without a baseline, we cannot say whether `GridSearchCV` or `DC Level` actually improved the model. Training Now the most important line. `model.fit(.)` Question. What happens here? This is where the SVM learns the relationship between the 17 features and the fault labels. Before this line, the model knows nothing. After this line, it has learned from the training data.

### PROFESSOR ASKS

## What does fit() do?

### EXCELLENT ANSWER

The `fit()` function trains the SVM by learning the optimal decision boundaries that separate the different fault classes using the training feature vectors and their corresponding labels. Complete Pipeline So Far Feature CSV

### Read Dataset

Separate X and Y

### Encode Labels

### Train-Test Split

**StandardScaler** — Create SVM.

### Train Model

Training is now complete. The next step is prediction and evaluation.

## COMMON MISTAKES

MISTAKE: [X] Scaling the entire dataset before splitting. Wrong. Always split first. Then fit only on training data.

MISTAKE: [X] Training on the test set. Wrong. The test set must remain unseen.

MISTAKE: [X] Forgetting Label Encoding. Wrong. SVM cannot train using text labels.

MISTAKE: [X] Comparing tuned models without a baseline. Wrong. Always establish a baseline first.

## REVISION NOTES

Remember the SVM training pipeline. **Read CSV** — Separate Features & Labels. **Encode Labels** — Train-Test Split. **StandardScaler** — Create SVM. Train This completes the learning phase.

**End of Chapter 26 (Part 1) Congratulations. You now understand the training stage of your baseline SVM implementation. You can confidently explain: Why features and labels are separated. Why Label Encoding is required. Why train-test splitting is essential. Why StandardScaler must be used. What fit() actually does. Why a baseline model is created before optimization. Author's Note The next continuation will cover Chapter 26 (Part 2): predict() Accuracy calculation Confusion Matrix Classification Report Precision Recall F1 Score Interpreting your baseline results Why the confusion matrix led to the discovery of the RC\_Open problem This is where your project transitions from training to scientific evaluation, and where the key engineering improvements begin.**

## PART 2 — Prediction & Model Evaluation

Chapter 26 — Complete Python Source Code Explained Part 2 — day14\_train\_svm.py (Prediction & Model Evaluation) (Based on your actual source code)

### Before Starting

#### In Part 1,

our SVM finished learning. The model has now seen the training data. Question. How do we know whether it actually learned something useful? Simple. We test it using unseen data. This chapter explains how your code evaluates the trained model. Where Are We? Our pipeline currently is

### Feature Dataset

**Train SVM** —???. Now we move to

### Test Dataset

**Prediction** — Evaluation. Performance Analysis Step 1 Making Predictions Your code uses `y_pred = model.predict(X_test)` Question. What happens here? The trained SVM receives the scaled test feature vectors. For each sample, it predicts one fault class. Example **Input** — [Peak, RMS, Mean,., DC Level].

**Prediction** — RC\_Open. This repeats for every test sample.

#### PROFESSOR ASKS

What does `predict()` do?

#### EXCELLENT ANSWER

The `predict()` function applies the trained SVM model to unseen test feature vectors and returns the predicted fault class for each sample.

### Step 2 — Comparing Predictions

Now we have two arrays.

#### Actual Labels

Normal Open Short Noise and

#### Predicted Labels

Normal Open Open Noise Question. How do we know whether the model performed well? We compare them.

### Step 3 — Accuracy

Your code computes `accuracy_score(.)` Formula  $\frac{\text{Correct Predictions}}{\text{Total Predictions}}$  Example Suppose 100 samples are tested. Model predicts 95 correctly. **Accuracy** — 95%. Very simple. Question. Is Accuracy enough? No. Very important.

#### PROFESSOR ASKS

Why is Accuracy alone insufficient?

#### EXCELLENT ANSWER

Accuracy provides only the overall percentage of correct predictions. It does not indicate which fault classes are being confused, making additional evaluation metrics necessary.

### Step 4 — Classification Report

Your code generates a `classification_report(.)` Question. What does it contain? For every class, it reports Precision Recall F1 Score Support Instead of one number, we now get a detailed analysis. Precision Formula  $\frac{\text{Correct Positive Predictions}}{\text{All Positive Predictions}}$  Question. Meaning? If the model predicts RC\_Open, Precision tells us how often that prediction is actually correct. Recall Formula  $\frac{\text{Correct Positive Predictions}}{\text{Actual Positive Samples}}$  Question. Meaning? How many real RC\_Open faults did the model successfully detect?

## F1 Score

Formula  $\frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$  Conceptually Balance between Precision and Recall Question. Why use F1? Because a model can have high Precision but low Recall, or vice versa. F1 combines both.

| PROFESSOR ASKS                       |
|--------------------------------------|
| What is the purpose of the F1 Score? |
| EXCELLENT ANSWER                     |
|                                      |

The F1 Score combines Precision and Recall into a single metric, providing a balanced measure of classification performance, especially when evaluating individual fault classes.

## Step 5 — Confusion Matrix

This is the most important evaluation tool in your project. Your code creates a `confusion_matrix(.)` Question. What is a Confusion Matrix? It is a table showing how predictions compare with the actual classes. Example Predicted

### N O S

Actual

N 20 0 0

O 1 18 1

S 0 2 18

**Diagonal** — Correct predictions.

**Off-diagonal** — Mistakes.

## Why Is It Called

Confusion Matrix? Because it tells us which classes the model is confusing. Example Suppose **RC\_Open** — Predicted. as RC\_Normal The model is confused between those classes. Your Project's Turning Point This is exactly what happened during your project. The confusion matrix revealed that RC\_Open was being misclassified. Instead of changing the SVM, you investigated the features. That investigation led to DC Level Topology Rule This is one of the strongest engineering decisions in your entire project.

| PROFESSOR ASKS                          |
|-----------------------------------------|
| Why was the confusion matrix important? |
| EXCELLENT ANSWER                        |
|                                         |

The confusion matrix identified recurring misclassification of RC\_Open faults. This guided feature engineering, leading to the introduction of the DC Level feature and later the Topology Rule.

## Step 6 — Printing Results

Your code prints the evaluation metrics to the console. This includes Accuracy Classification Report Confusion Matrix These outputs allow you to judge whether the baseline model is satisfactory. Is the

Baseline Perfect? No. Question. Why? Because this is only the first model. Its purpose is to establish a reference point. Later, you improve it using GridSearchCV DC Level Topology Rule Without the baseline, you cannot prove that those improvements actually worked. Complete Evaluation Pipeline **Scaled Test Data** — Predict().

### Predicted Labels

**Compare with Actual Labels** — Accuracy.

### Classification Report

### Confusion Matrix

### Analyze Errors

This completes the evaluation stage.

### Engineering Mindset

Notice what you did. Many students see an error and immediately change the algorithm. You didn't. You first asked why the error occurred. That is engineering.

## COMMON MISTAKES

MISTAKE: [X] Looking only at Accuracy. Wrong. Always examine the Confusion Matrix.

MISTAKE: [X] Ignoring low Recall. Wrong. Missing actual faults can be very dangerous.

MISTAKE: [X] Optimizing before understanding the errors. Wrong. Analyze first. Improve later.

MISTAKE: [X] Treating the baseline as the final model. Wrong. It is only the starting point.

## REVISION NOTES

Remember the evaluation pipeline. **Predict** — Accuracy. **Classification Report** — Confusion Matrix. **Error Analysis** — Engineering Improvements. Memory Trick Remember this sentence. "Train, Predict, Measure, Analyze, Improve." These five words describe the entire machine learning workflow.

## KNOWLEDGE CHECK

### Level 1

- What does predict() do?
- How is Accuracy calculated?
- What is Precision?
- What is Recall?
- Why is the Confusion Matrix important?

### Level 2

- Why is Accuracy not enough?
- Explain the difference between Precision and Recall.
- How did the Confusion Matrix influence your project?
- Why was the baseline model necessary?
- Describe the complete evaluation pipeline.

### Professor Level



- Explain how your baseline SVM was evaluated.
- Why is class-wise evaluation more informative than overall Accuracy?
- How did error analysis lead to feature engineering improvements?
- Why should engineering decisions be guided by data rather than assumptions?
- What was the biggest insight obtained from your baseline evaluation?
- End of Chapter 26 (Part 2)
- Congratulations.
- You now understand the complete baseline SVM workflow:
- Training.
- Prediction.
- Accuracy evaluation.
- Classification Report.
- Confusion Matrix.
- Error analysis.
- More importantly,
- you understand why your project improved.
- The improvements were not random.
- They were driven by evidence from the Confusion Matrix, which led to better features and smarter engineering decisions.

Author's Note The next continuation begins Chapter 27 — `day15_gridsearch_svm.py`. We'll explain: Why the baseline SVM wasn't enough. Hyperparameters ( $C$ ,  $\gamma$ , kernel). What GridSearchCV actually does. Cross-validation (5-fold) explained visually. How the best model is selected. Why GridSearchCV is better than manual tuning. Every important line of your tuning code. This chapter will explain how your baseline model evolved into the optimized classifier used in the final project.